Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai) Bhavan's Campus, Munshi Nagar,
Andheri (West), Mumbai-400058-India

Department of Computer Engineering · DBMS Mini Project (CECS202)

Phase II — Complex SQL Queries: Joins, Aggregates & Subqueries (Exp 8)

Project: Surplus Food Rescue | Batch: B2

Sameel Kazi (2025300107) · Pradhyun Kembhavi (2025300109) · Adeeb Khan (2025300110)

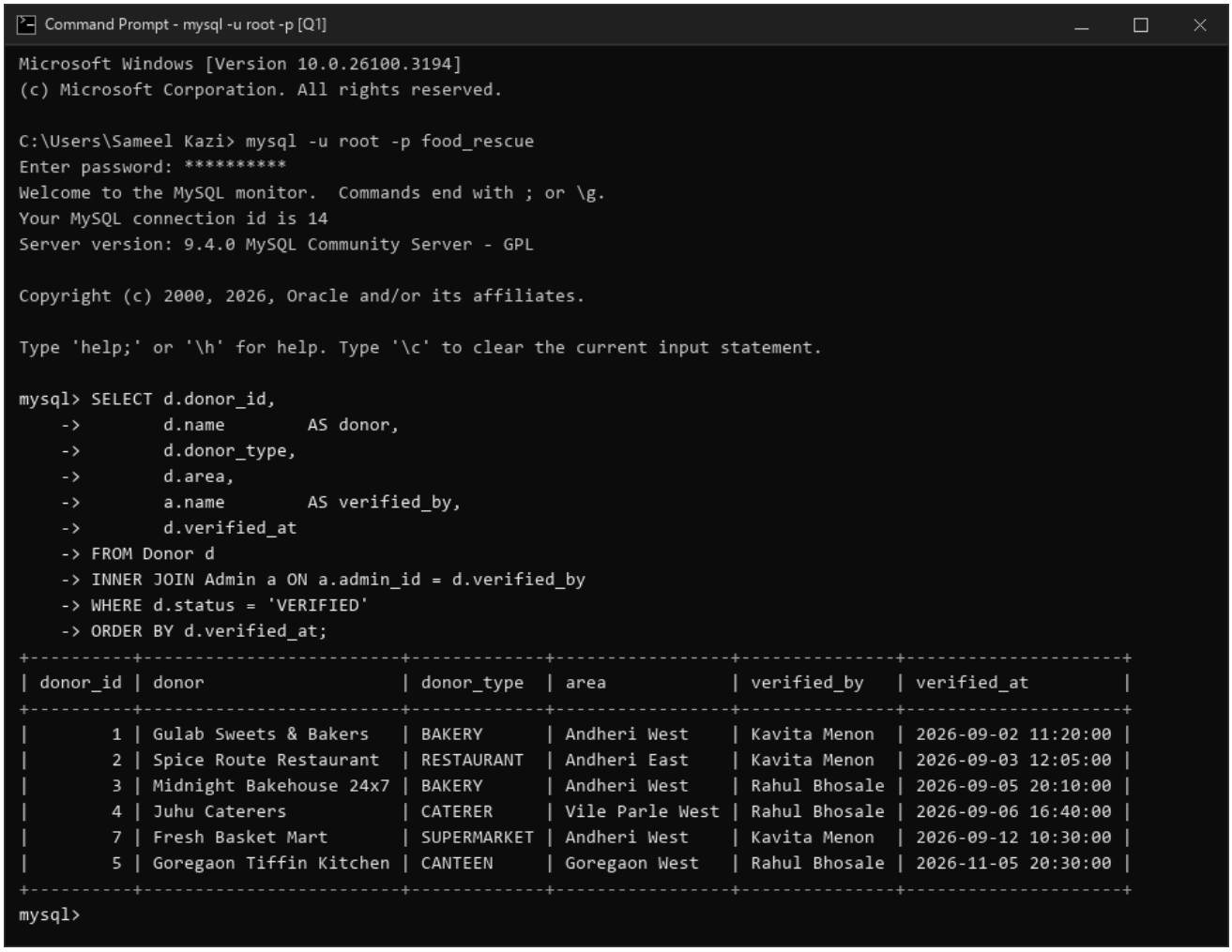
Note on Query Execution Context: All queries are executed on the normalized 'food_rescue' database with a standardized demo clock SET @at = '2026-11-05 20:30:00' (Thursday evening rush) and GPS coordinates @my_lat = 19.1215, @my_lng = 72.8348 (Andheri West, Mumbai). This ensures perfectly deterministic and verifiable results matching all sample batches.

Part A. Relational Joins (Queries Q1 to Q7)

Q1: Verified Donors and Verifying Admin

Objective: List all active food rescue donors and the municipal admin staff member who verified their FSSAI registration. **Explanation:** Demonstrates an INNER JOIN between Donor and Admin on a.admin_id = d.verified_by, filtering strictly for verified establishments.

```
SELECT d.donor_id, d.name AS donor, d.donor_type, d.area, a.name AS verified_by, d.verified_at
FROM Donor d
INNER JOIN Admin a ON a.admin_id = d.verified_by
WHERE d.status = 'VERIFIED'
ORDER BY d.verified_at;
```



MySQL Output: Q1 execution result

Q2: Donor Review Status Audit

Objective: Inspect all registered food businesses and audit whether they have been reviewed, and by whom. **Explanation:** Uses a LEFT JOIN between Donor and Admin with COALESCE to capture pending/unassigned donors without losing unreviewed rows.

```
SELECT d.name AS donor, d.status, COALESCE(a.name, 'Not reviewed yet') AS reviewed_by
FROM Donor d
LEFT JOIN Admin a ON a.admin_id = d.verified_by
ORDER BY FIELD(d.status, 'PENDING', 'REJECTED', 'VERIFIED'), d.name;
```

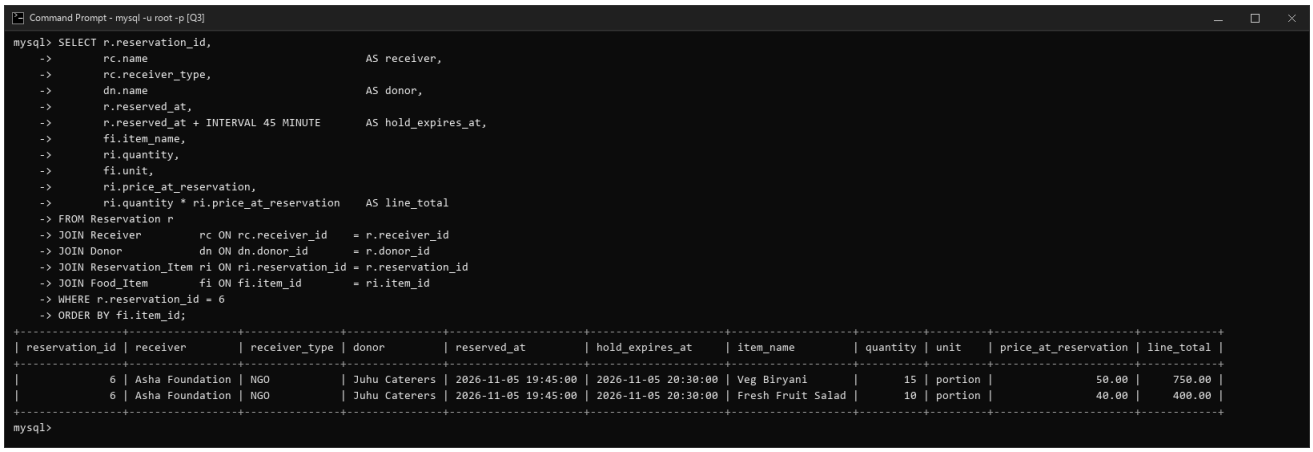


MySQL Output: Q2 execution result

Q3: Complete Reservation Receipt & Breakdown

Objective: Display a comprehensive receipt for reservation #6, showing receiver, donor, item breakdown, and computed 45-minute hold expiry. **Explanation:** Performs a five-table join (Reservation, Receiver, Donor, Reservation_Item, Food_Item) with a derived datetime attribute.

```
SELECT r.reservation_id, rc.name AS receiver, rc.receiver_type, dn.name AS donor,
       r.reserved_at, r.reserved_at + INTERVAL 45 MINUTE AS hold_expires_at,
       fi.item_name, ri.quantity, fi.unit, ri.price_at_reservation,
       ri.quantity * ri.price_at_reservation AS line_total
FROM Reservation r
JOIN Receiver rc ON rc.receiver_id = r.receiver_id
JOIN Donor dn ON dn.donor_id = r.donor_id
JOIN Reservation_Item ri ON ri.reservation_id = r.reservation_id
JOIN Food_Item fi ON fi.item_id = ri.item_id
WHERE r.reservation_id = 6;
```

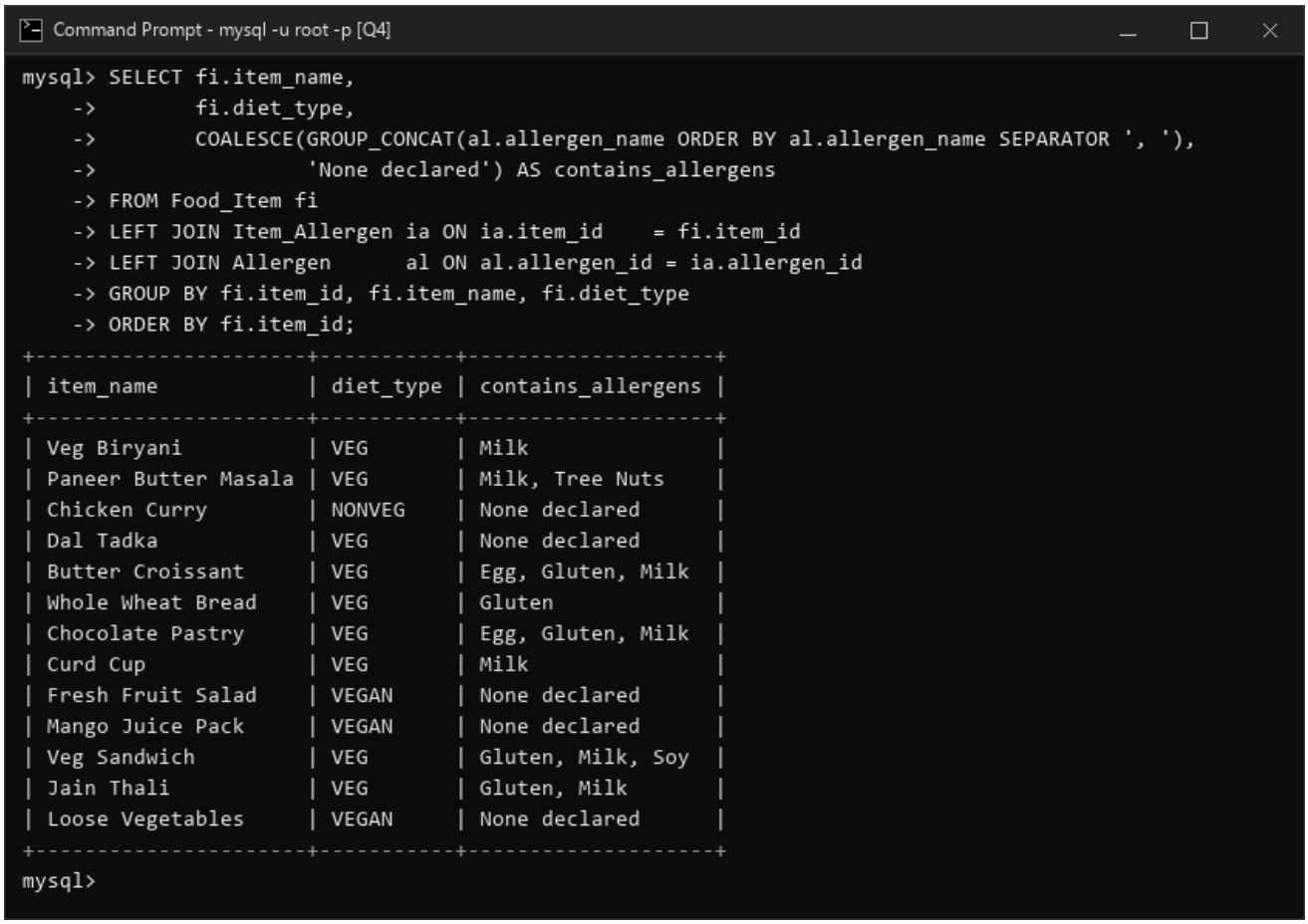


MySQL Output: Q3 execution result

Q4: Allergen Transparency Sheet

Objective: Produce an allergen safety report for every catalog food item for receiver health compliance. **Explanation:** Resolves the M:N relationship through junction table Item_Allergen and aggregates tags using GROUP_CONCAT.

```
SELECT fi.item_name, fi.category, fi.diet_type,
       COALESCE(GROUP_CONCAT(al.allergen_name ORDER BY al.allergen_name SEPARATOR ', '),
                'None declared') AS contains_allergens
FROM Food_Item fi
LEFT JOIN Item_Allergen ia ON ia.item_id = fi.item_id
LEFT JOIN Allergen al ON al.allergen_id = ia.allergen_id
GROUP BY fi.item_id, fi.item_name, fi.category, fi.diet_type;
```

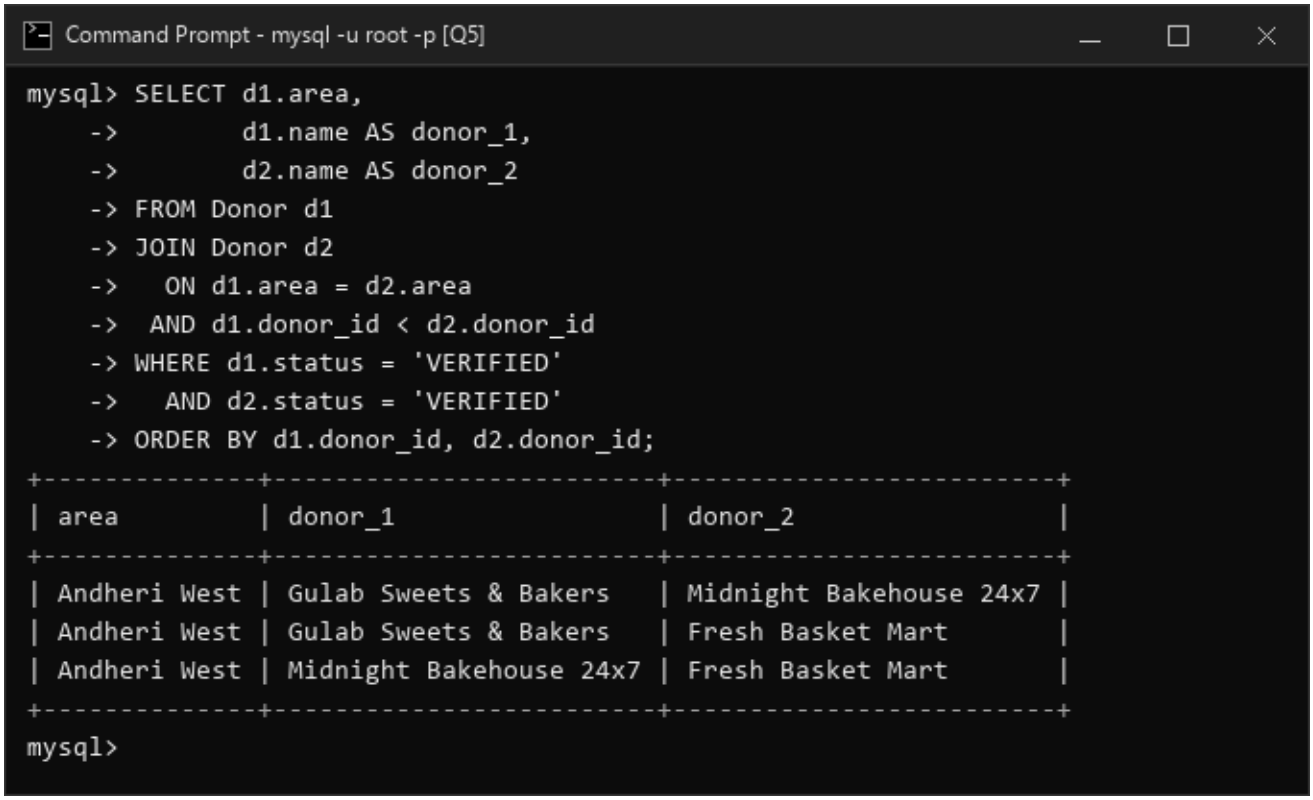


MySQL Output: Q4 execution result

Q5: Colocated Donors in the Same Locality

Objective: Identify pairs of verified food donors in the same neighborhood for joint pickup logistics. **Explanation:** Implements a SELF JOIN on Donor d1 and Donor d2 matching area with strict inequality d1.donor_id < d2.donor_id to prevent duplicate or reflexive pairs.

```
SELECT d1.area, d1.name AS donor_1, d2.name AS donor_2
FROM Donor d1
JOIN Donor d2 ON d1.area = d2.area AND d1.donor_id < d2.donor_id
WHERE d1.status = 'VERIFIED' AND d2.status = 'VERIFIED'
ORDER BY d1.donor_id, d2.donor_id;
```

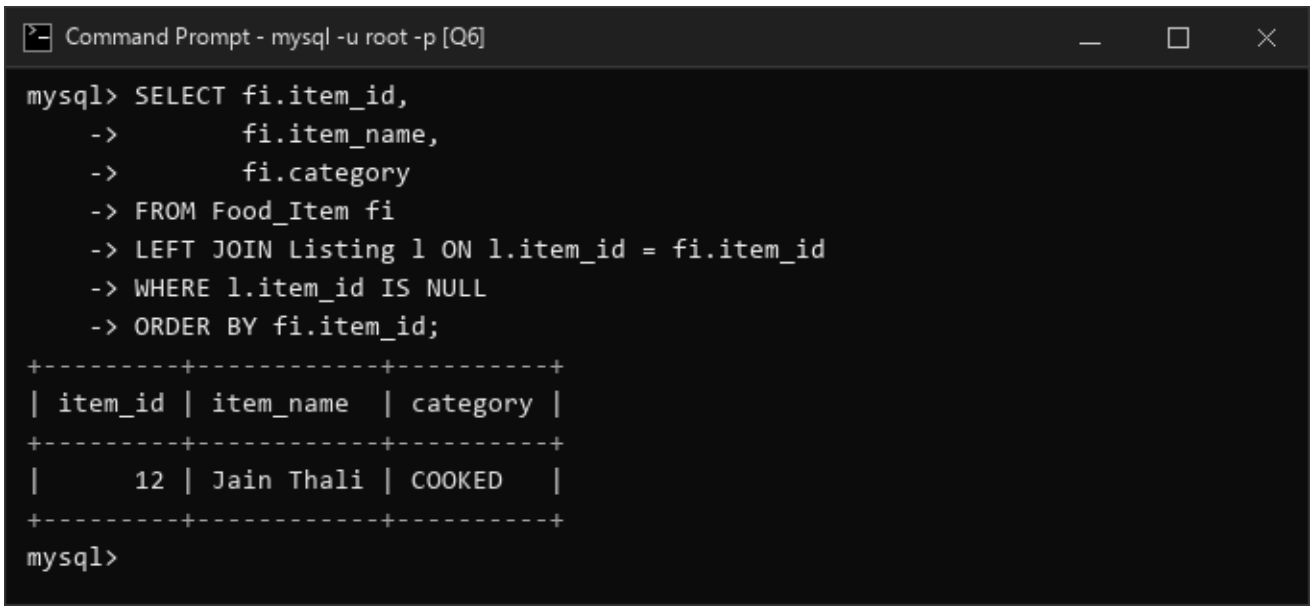


MySQL Output: Q5 execution result

Q6: Unlisted Food Items (Anti-Join)

Objective: Find food items in the central catalog that no active donor has ever listed for rescue. **Explanation:** Employs an anti-join pattern: LEFT JOIN Listing l ON l.item_id = fi.item_id WHERE l.item_id IS NULL.

```
SELECT fi.item_id, fi.item_name, fi.category
FROM Food_Item fi
LEFT JOIN Listing l ON l.item_id = fi.item_id
WHERE l.item_id IS NULL
ORDER BY fi.item_id;
```

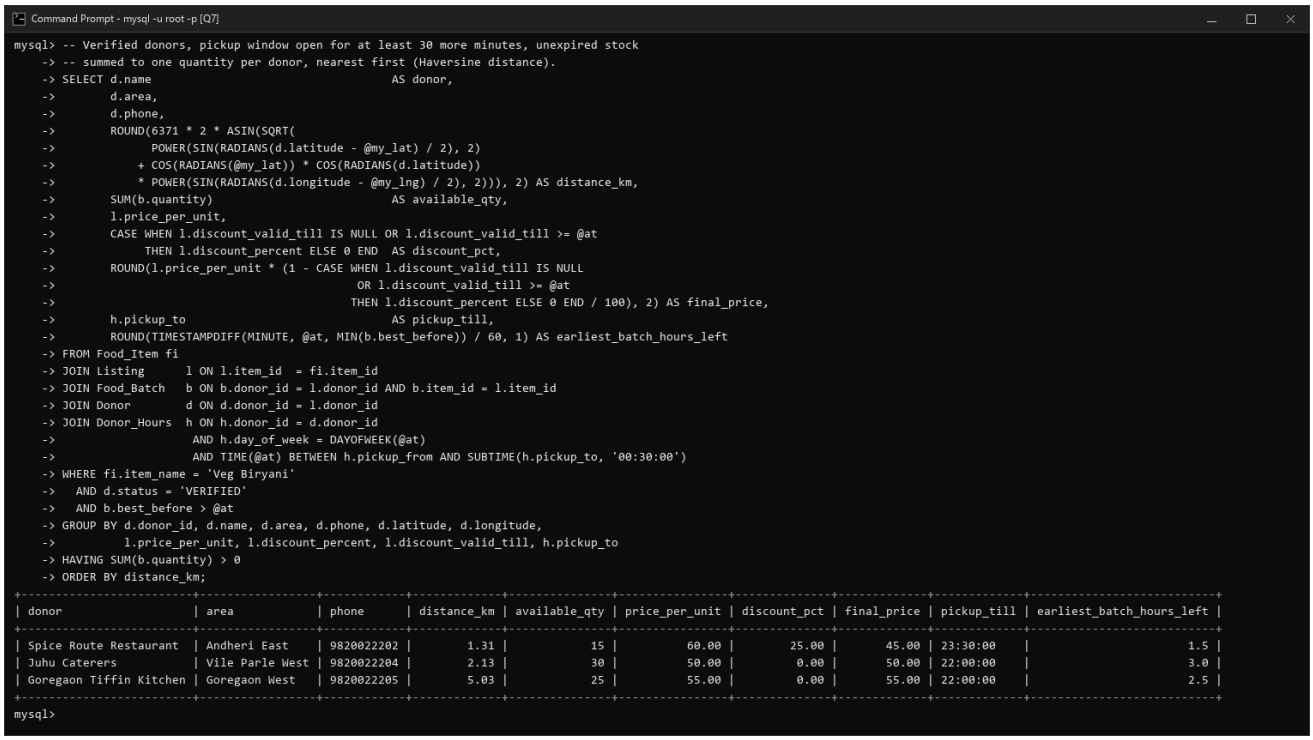


MySQL Output: Q6 execution result

Q7: Core Surplus Search: Nearest Open Donors with Safe Stock

Objective: Find where a receiver standing at Andheri West can get 'Veg Biryani' right now within open pickup hours. **Explanation:** A 5-table join computing Haversine radial distance, summing non-expired batches, applying flash discounts, and verifying pickup window validity.

```
SELECT d.name AS donor, d.area, d.phone,
       ROUND(6371 * 2 * ASIN(SQRT(
             POWER(SIN(RADIANS(d.latitude - @my_lat) / 2), 2)
           + COS(RADIANS(@my_lat)) * COS(RADIANS(d.latitude))
           * POWER(SIN(RADIANS(d.longitude - @my_lng) / 2), 2))), 2) AS distance_km,
       SUM(b.quantity) AS available_qty, l.price_per_unit,
       ROUND(l.price_per_unit * (1 - CASE WHEN l.discount_valid_till IS NULL OR l.discount_valid_till >= @at
                                          THEN l.discount_percent ELSE 0 END / 100), 2) AS final_price,
       h.pickup_to AS pickup_till,
       ROUND(TIMESTAMPDIFF(MINUTE, @at, MIN(b.best_before)) / 60, 1) AS earliest_batch_hours_left
FROM Food_Item fi
JOIN Listing l ON l.item_id = fi.item_id
JOIN Food_Batch b ON b.donor_id = l.donor_id AND b.item_id = l.item_id
JOIN Donor d ON d.donor_id = l.donor_id
JOIN Donor_Hours h ON h.donor_id = d.donor_id AND h.day_of_week = DAYOFWEEK(@at)
                  AND TIME(@at) BETWEEN h.pickup_from AND SUBTIME(h.pickup_to, '00:30:00')
WHERE fi.item_name = 'Veg Biryani' AND d.status = 'VERIFIED' AND b.best_before > @at
GROUP BY d.donor_id, d.name, d.area, d.phone, d.latitude, d.longitude,
         l.price_per_unit, l.discount_percent, l.discount_valid_till, h.pickup_to
HAVING SUM(b.quantity) > 0
ORDER BY distance_km;
```



MySQL Output: Q7 execution result

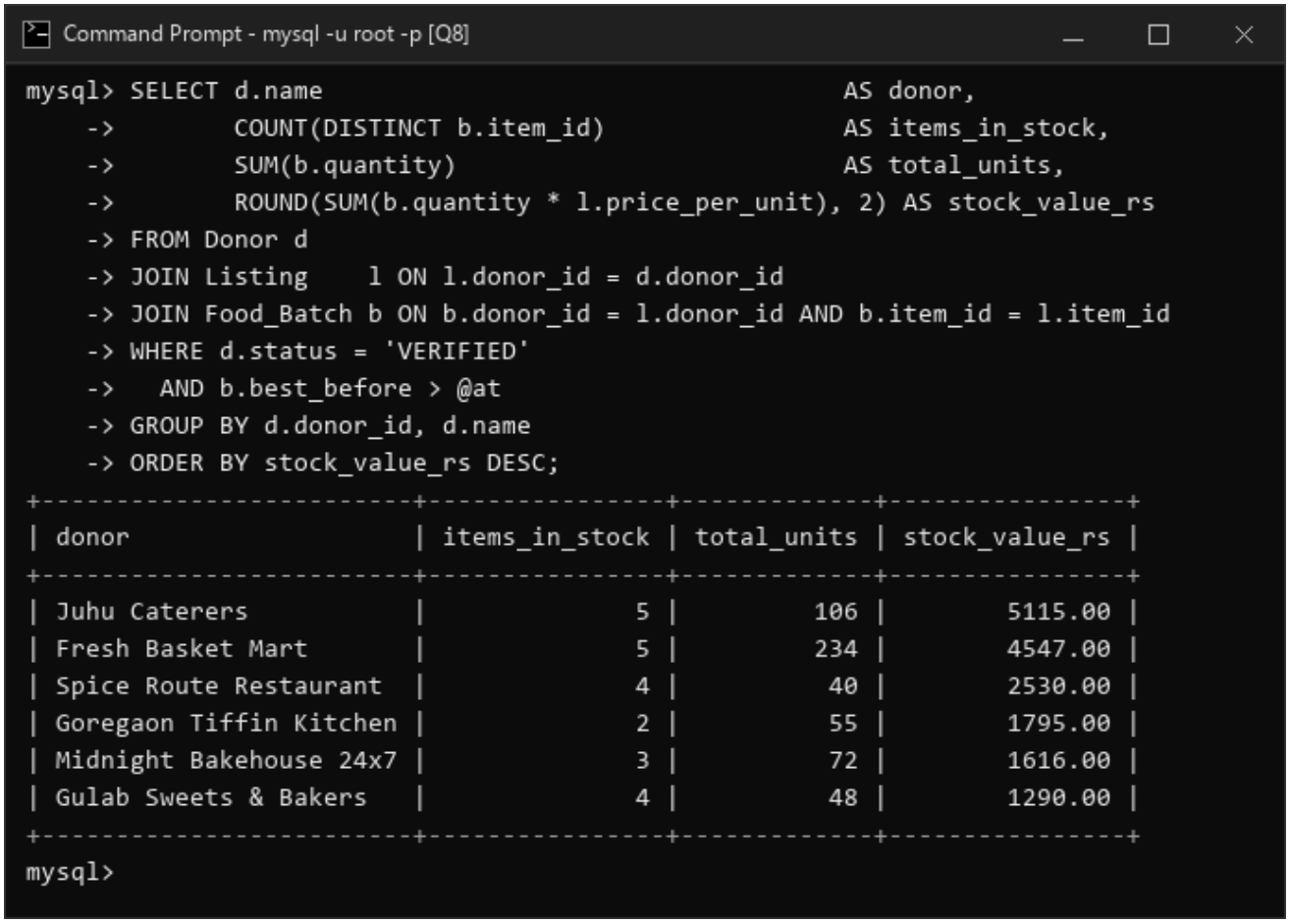
Part B. Aggregate Functions & GROUP BY (Queries Q8 to Q13)

Q8: Live Inventory Valuation per Donor

Objective: Summarize unexpired rescue stock items, unit totals, and gross commercial value by donor. **Explanation:** Combines COUNT(DISTINCT), SUM, and computed monetary valuation filtered for unexpired batches.

```
SELECT d.name AS donor, COUNT(DISTINCT b.item_id) AS items_in_stock,
       SUM(b.quantity) AS total_units,
       ROUND(SUM(b.quantity * l.price_per_unit), 2) AS stock_value_rs
```

```
FROM Donor d
JOIN Listing l ON l.donor_id = d.donor_id
JOIN Food_Batch b ON b.donor_id = l.donor_id AND b.item_id = l.item_id
WHERE d.status = 'VERIFIED' AND b.best_before > @at
GROUP BY d.donor_id, d.name
ORDER BY stock_value_rs DESC;
```

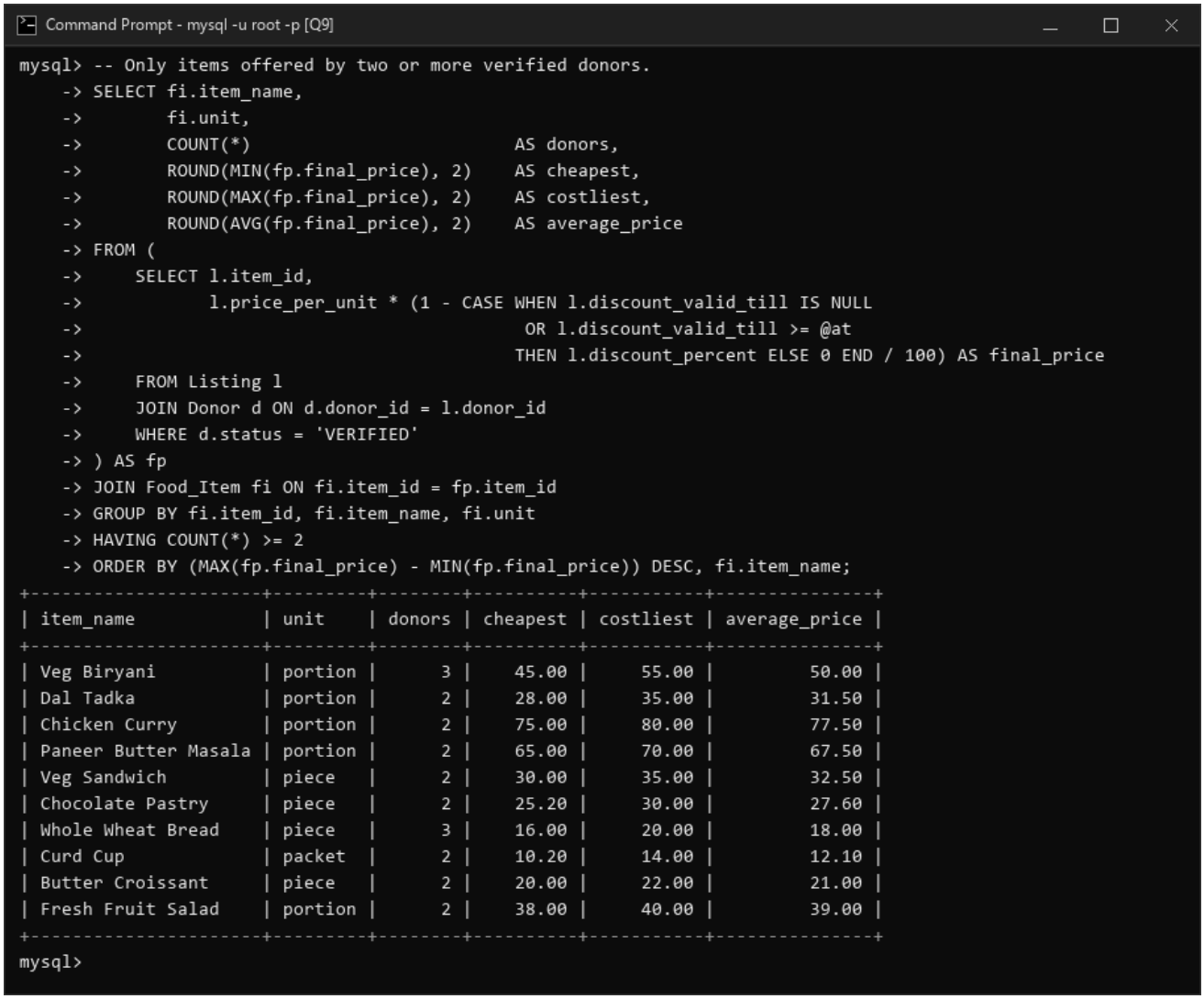


MySQL Output: Q8 execution result

Q9: Price Dispersion & Item Comparison

Objective: Compare price spread (MIN, MAX, AVG final price) for items offered by multiple donors. **Explanation:** Aggregates over a subquery calculating discounted pricing, applying HAVING COUNT(*) >= 2 to isolate competitive food items.

```
SELECT fi.item_name, fi.unit, COUNT(*) AS donors,
       ROUND(MIN(fp.final_price), 2) AS cheapest,
       ROUND(MAX(fp.final_price), 2) AS costliest,
       ROUND(AVG(fp.final_price), 2) AS average_price
FROM (
    SELECT l.item_id,
           l.price_per_unit * (1 - CASE WHEN l.discount_valid_till IS NULL OR l.discount_valid_till >= @at
                                        THEN l.discount_percent ELSE 0 END / 100) AS final_price
    FROM Listing l JOIN Donor d ON d.donor_id = l.donor_id WHERE d.status = 'VERIFIED'
) AS fp
JOIN Food_Item fi ON fi.item_id = fp.item_id
GROUP BY fi.item_id, fi.item_name, fi.unit
HAVING COUNT(*) >= 2
ORDER BY (MAX(fp.final_price) - MIN(fp.final_price)) DESC;
```

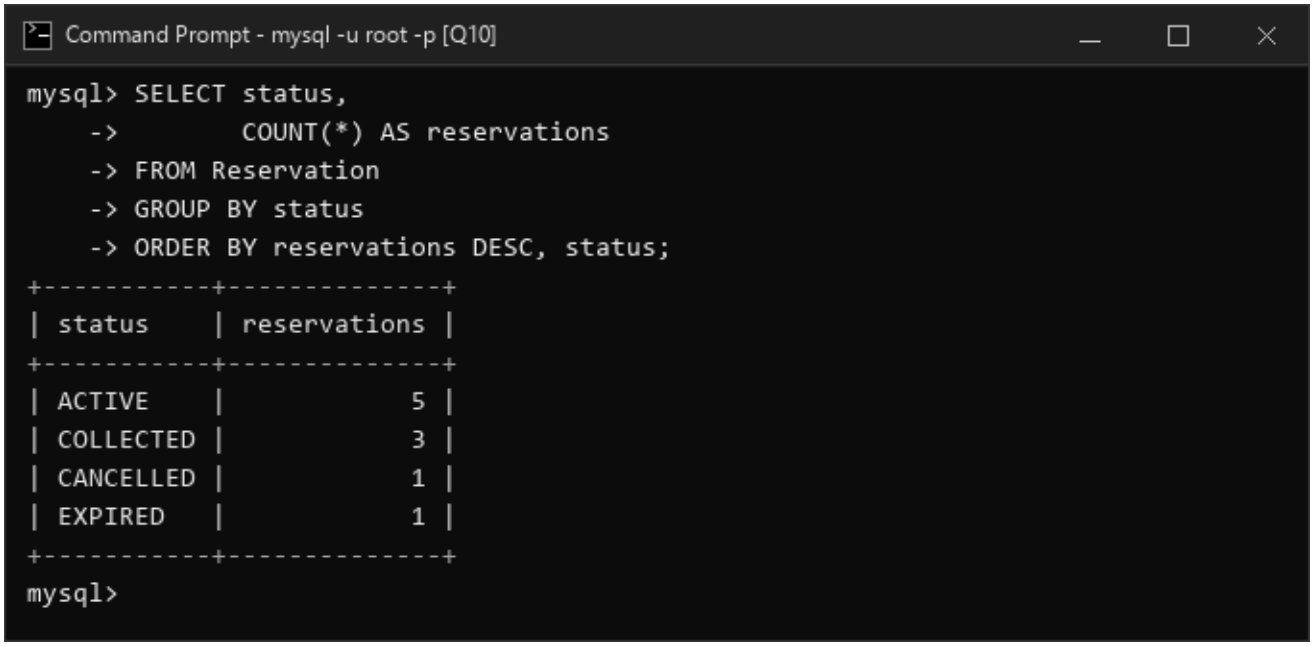


MySQL Output: Q9 execution result

Q10: Reservation Pipeline Breakdown

Objective: Count reservations across all operational statuses (COLLECTED, ACTIVE, CANCELLED, EXPIRED). **Explanation:** GROUP BY on categorical status column with COUNT(*) and deterministic ordering.

```
SELECT status, COUNT(*) AS reservations
FROM Reservation
GROUP BY status
ORDER BY reservations DESC, status;
```

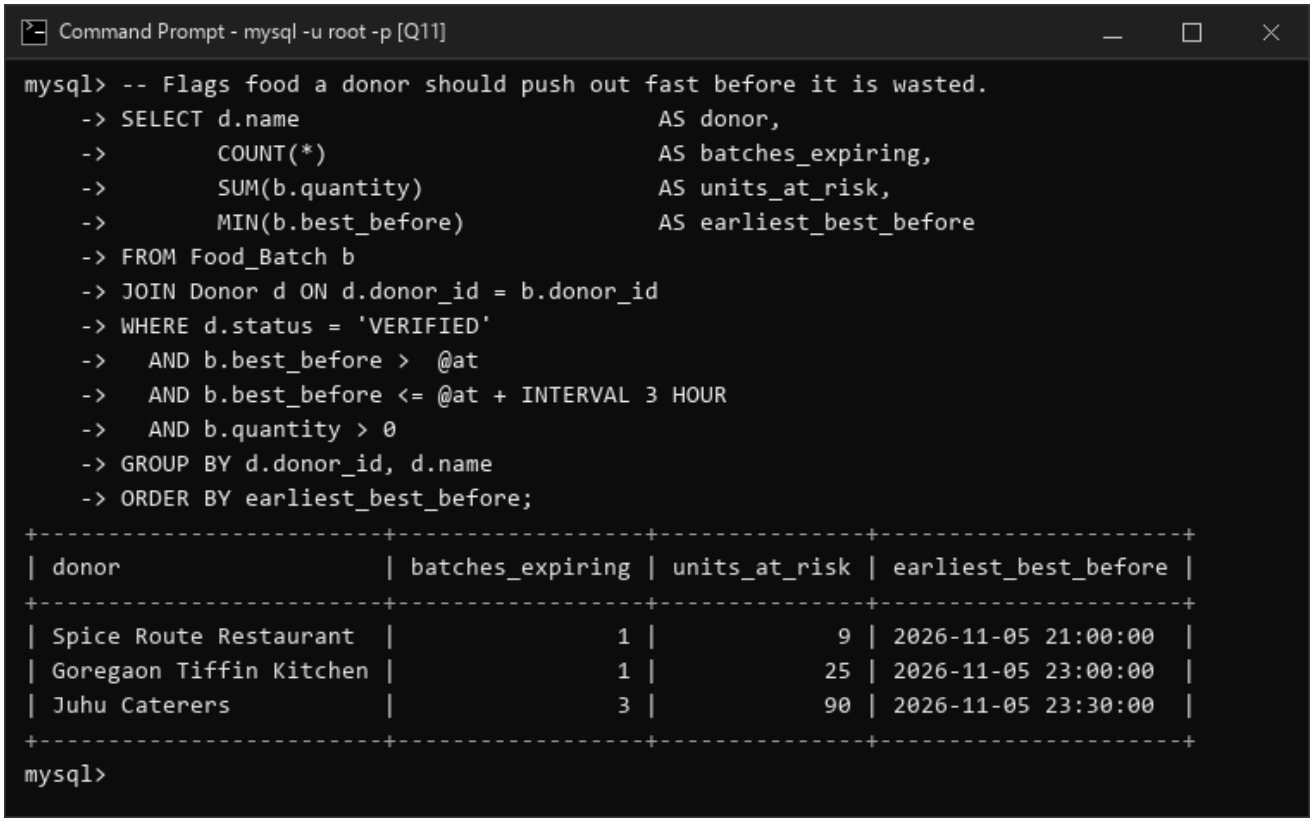


MySQL Output: Q10 execution result

Q11: Imminent Expiry Alert (< 3 Hours Left)

Objective: Detect perishable batches expiring within 3 hours so donors can initiate flash discounts or emergency donation. **Explanation:** Filters best-before within @at AND @at + INTERVAL 3 HOUR with MIN timestamp and SUM quantity.

```
SELECT d.name AS donor, COUNT(*) AS batches_expiring,
       SUM(b.quantity) AS units_at_risk, MIN(b.best_before) AS earliest_best_before
FROM Food_Batch b
JOIN Donor d ON d.donor_id = b.donor_id
WHERE d.status = 'VERIFIED' AND b.best_before > @at AND b.best_before <= @at + INTERVAL 3 HOUR
  AND b.quantity > 0
GROUP BY d.donor_id, d.name
ORDER BY earliest_best_before;
```

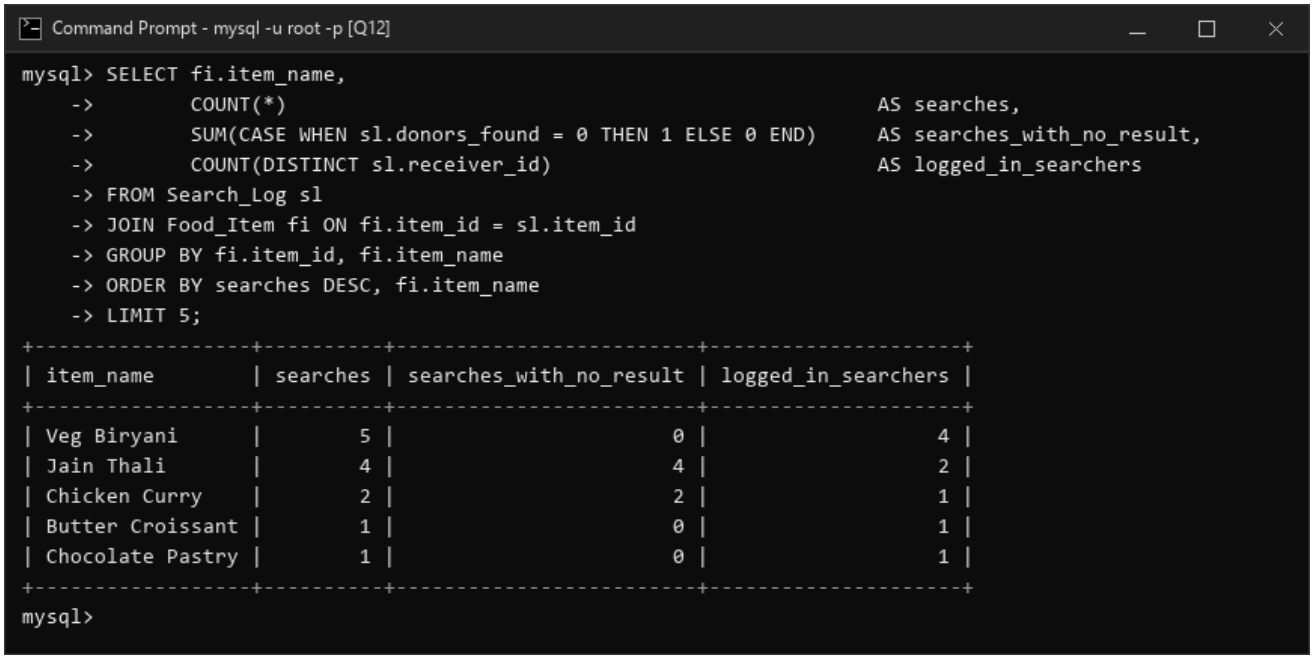


MySQL Output: Q11 execution result

Q12: Search Demand and Unmet Queries

Objective: Rank most sought-after food items and identify searches yielding zero results. **Explanation:** Aggregates Search_Log with conditional summation SUM(CASE WHEN donors_found = 0 THEN 1 ELSE 0 END).

```
SELECT fi.item_name, COUNT(*) AS searches,
       SUM(CASE WHEN sl.donors_found = 0 THEN 1 ELSE 0 END) AS searches_with_no_result,
       COUNT(DISTINCT sl.receiver_id) AS logged_in_searchers
FROM Search_Log sl
JOIN Food_Item fi ON fi.item_id = sl.item_id
GROUP BY fi.item_id, fi.item_name
ORDER BY searches DESC, fi.item_name
LIMIT 5;
```



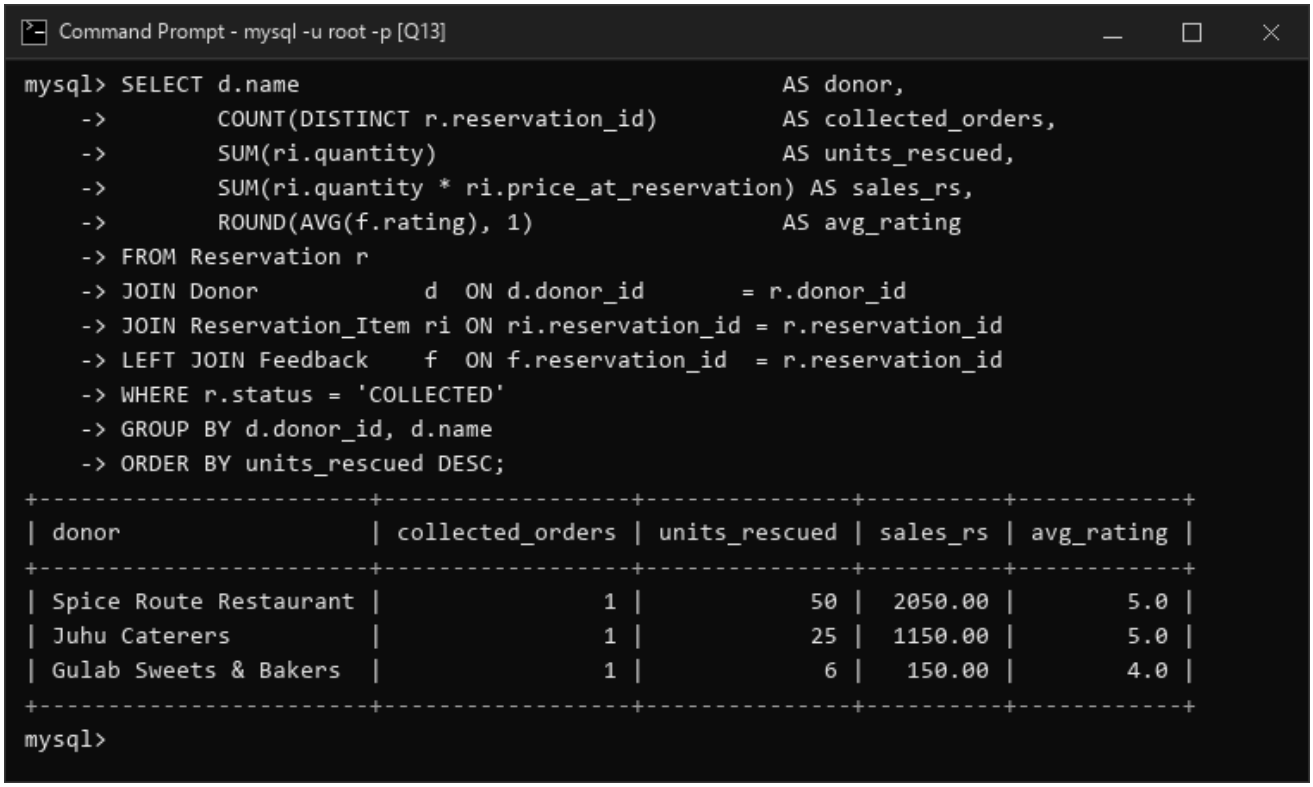
MySQL Output: Q12 execution result

Q13: Rescue Volume, Revenue & Feedback Rating

Objective: Summarize successfully collected orders, rescued food portions, total revenue, and average star ratings per donor.

Explanation: Joins Reservation, Reservation_Item, and Feedback for COLLECTED status with AVG(rating).

```
SELECT d.name AS donor, COUNT(DISTINCT r.reservation_id) AS collected_orders,
       SUM(ri.quantity) AS units_rescued,
       SUM(ri.quantity * ri.price_at_reservation) AS sales_rs,
       ROUND(AVG(f.rating), 1) AS avg_rating
FROM Reservation r
JOIN Donor d ON d.donor_id = r.donor_id
JOIN Reservation_Item ri ON ri.reservation_id = r.reservation_id
LEFT JOIN Feedback f ON f.reservation_id = r.reservation_id
WHERE r.status = 'COLLECTED'
GROUP BY d.donor_id, d.name
ORDER BY units_rescued DESC;
```



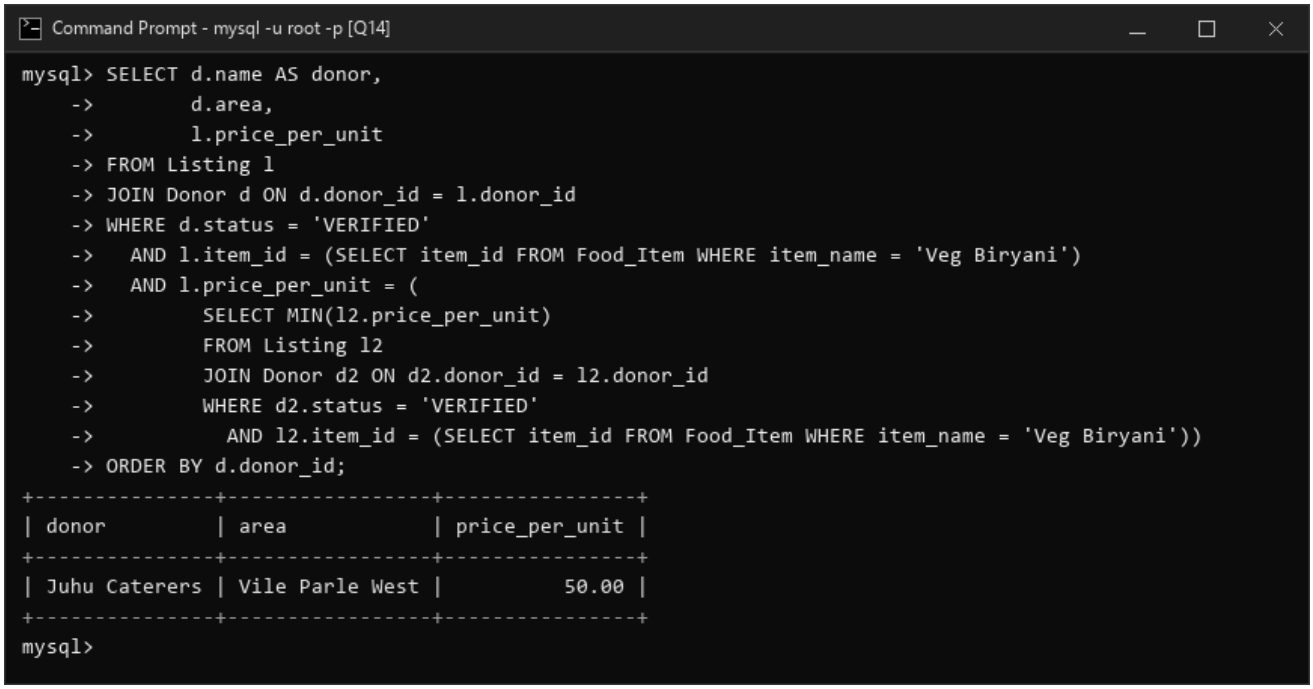
MySQL Output: Q13 execution result

Part C. Subqueries & Advanced SQL (Queries Q14 to Q20)

Q14: Cheapest Listing (Scalar Subquery)

Objective: Pinpoint the absolute cheapest verified donor listing for Veg Biryani. **Explanation:** Uses two nested scalar subqueries in WHERE: one for item_id and another for MIN(price_per_unit).

```
SELECT d.name AS donor, d.area, l.price_per_unit
FROM Listing l
JOIN Donor d ON d.donor_id = l.donor_id
WHERE d.status = 'VERIFIED'
  AND l.item_id = (SELECT item_id FROM Food_Item WHERE item_name = 'Veg Biryani')
  AND l.price_per_unit = (
        SELECT MIN(l2.price_per_unit)
        FROM Listing l2 JOIN Donor d2 ON d2.donor_id = l2.donor_id
        WHERE d2.status = 'VERIFIED' AND l2.item_id = (SELECT item_id FROM Food_Item WHERE item_name = 'Veg
Biryani'));
```

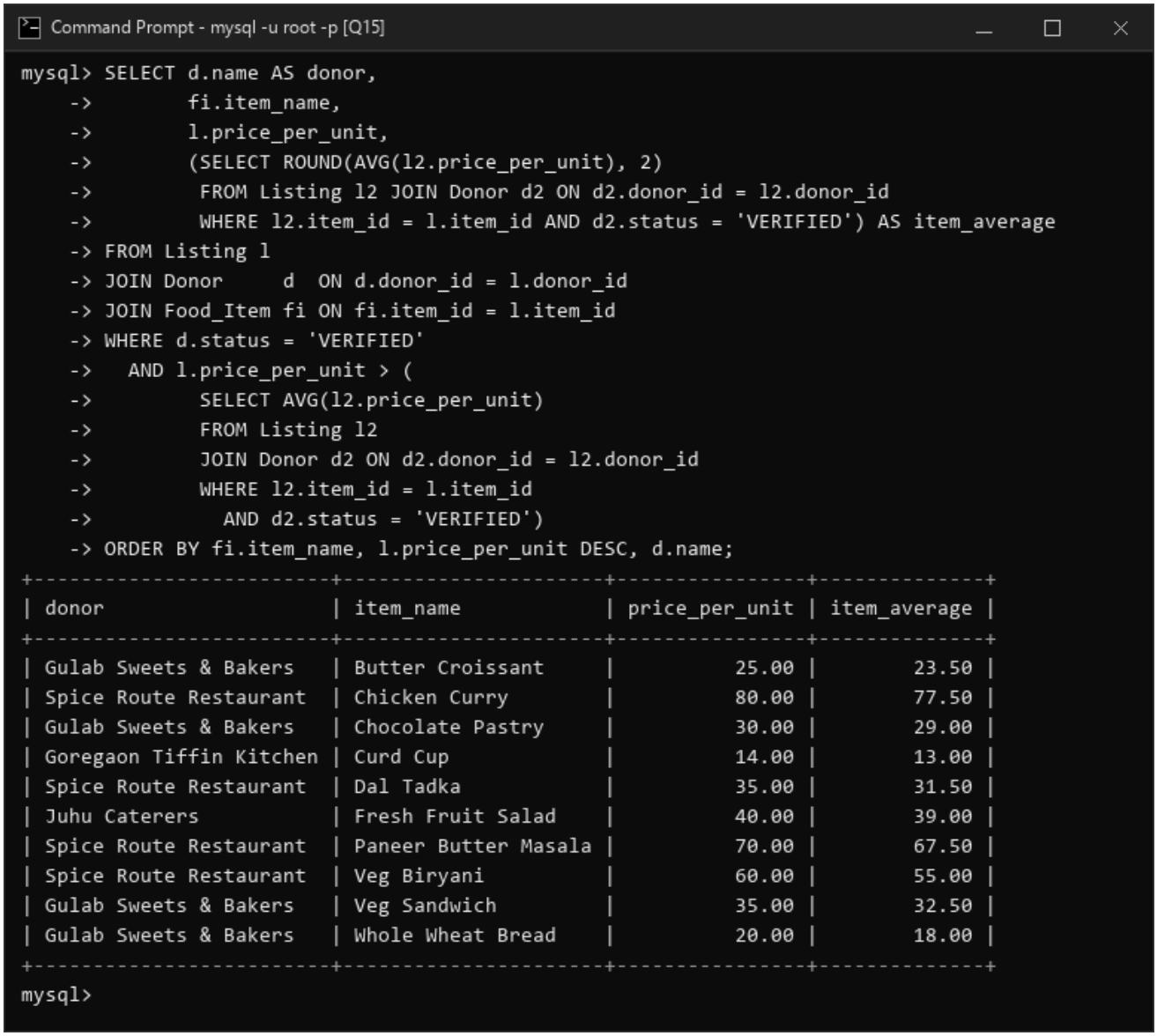


MySQL Output: Q14 execution result

Q15: Premium-Priced Listings (Correlated Subquery)

Objective: Find donor listings whose base price exceeds the city-wide average price for that specific item. **Explanation:** Employs a correlated subquery in WHERE that dynamically re-computes AVG(price_per_unit) for outer row l.item_id.

```
SELECT d.name AS donor, fi.item_name, l.price_per_unit,
       (SELECT ROUND(AVG(l2.price_per_unit), 2)
        FROM Listing l2 JOIN Donor d2 ON d2.donor_id = l2.donor_id
        WHERE l2.item_id = l.item_id AND d2.status = 'VERIFIED') AS item_average
FROM Listing l
JOIN Donor d ON d.donor_id = l.donor_id
JOIN Food_Item fi ON fi.item_id = l.item_id
WHERE d.status = 'VERIFIED'
  AND l.price_per_unit > (
       SELECT AVG(l2.price_per_unit)
       FROM Listing l2 JOIN Donor d2 ON d2.donor_id = l2.donor_id
       WHERE l2.item_id = l.item_id AND d2.status = 'VERIFIED')
ORDER BY fi.item_name, l.price_per_unit DESC;
```

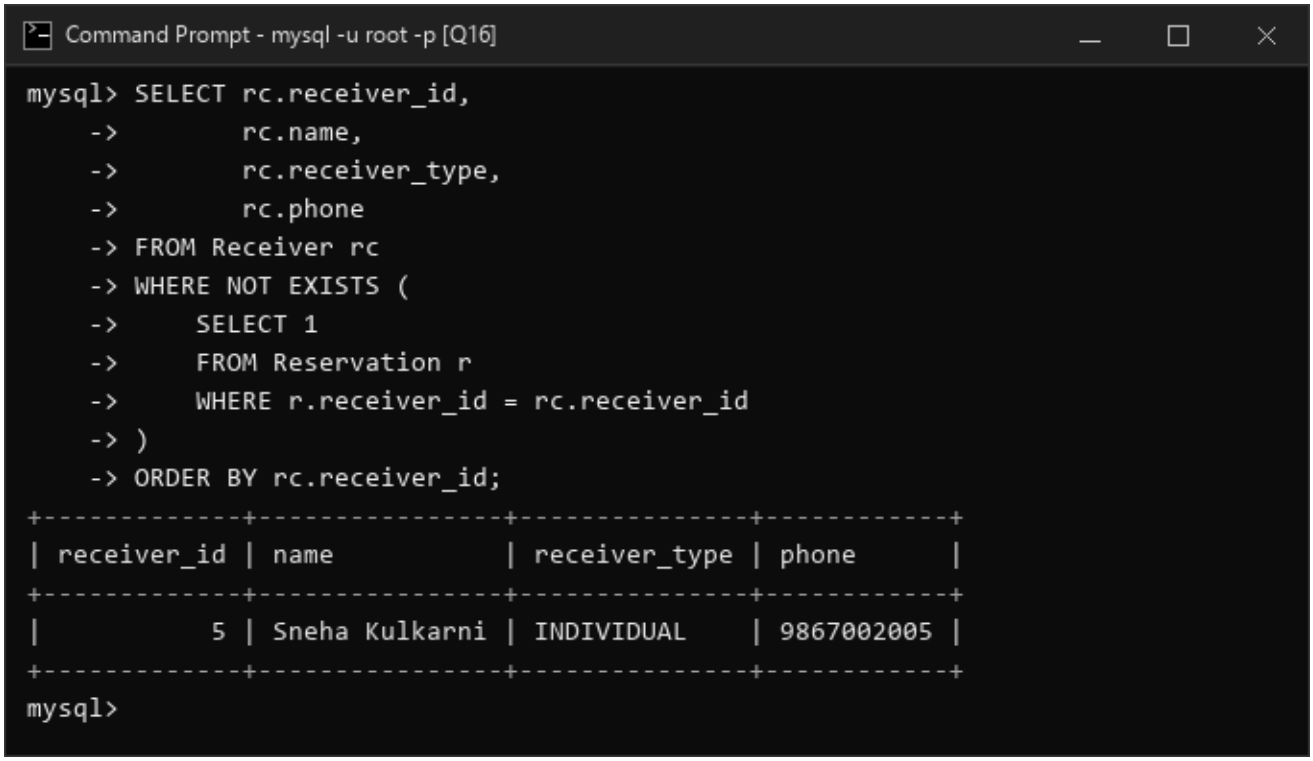


MySQL Output: Q15 execution result

Q16: Inactive Receivers (NOT EXISTS Subquery)

Objective: Identify registered beneficiaries who have never submitted any food reservation hold. **Explanation:** Demonstrates correlated NOT EXISTS subquery testing absence of matching rows in Reservation.

```
SELECT rc.receiver_id, rc.name, rc.receiver_type, rc.phone
FROM Receiver rc
WHERE NOT EXISTS (
    SELECT 1 FROM Reservation r WHERE r.receiver_id = rc.receiver_id
)
ORDER BY rc.receiver_id;
```

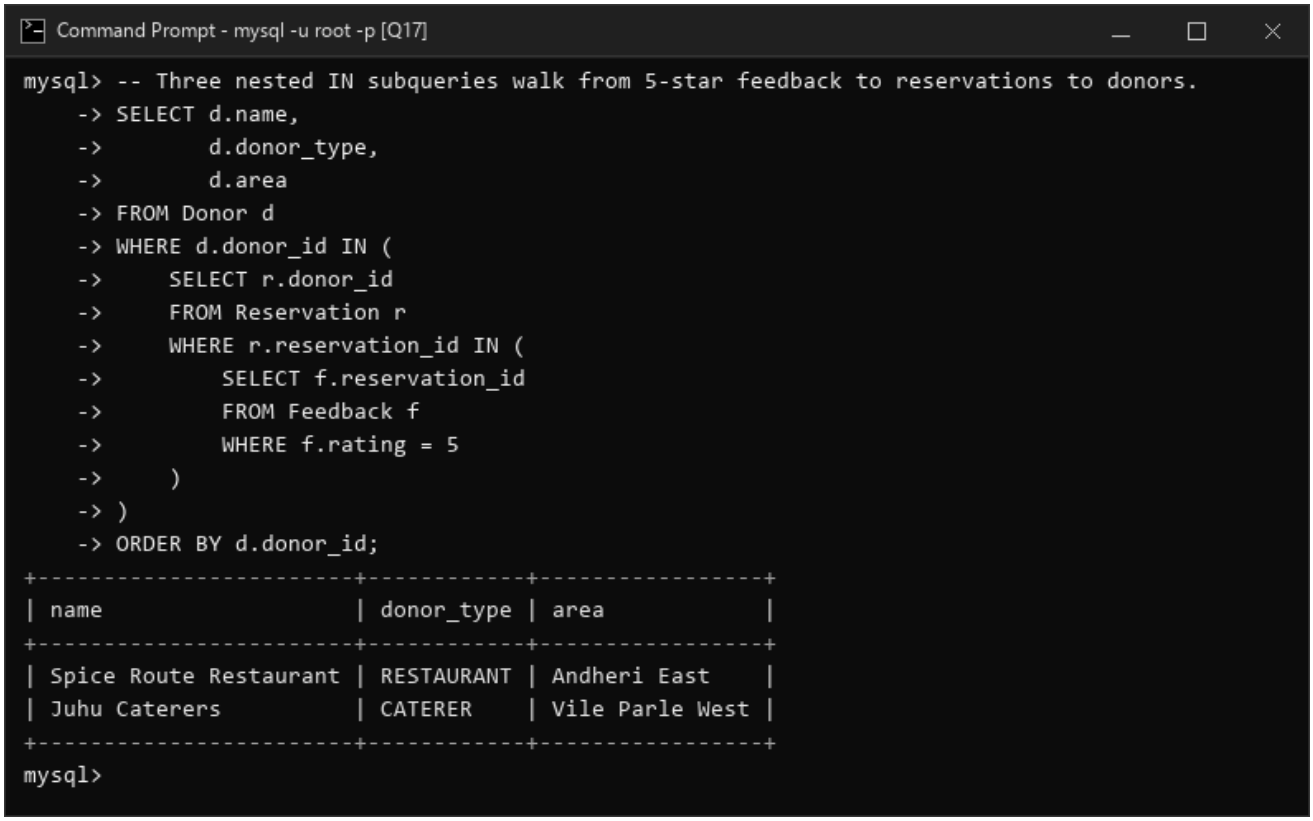


MySQL Output: Q16 execution result

Q17: Top-Rated Donors (Triple-Nested IN)

Objective: List donors who have received a perfect 5-star customer feedback review. **Explanation:** Three-level nested IN subquery: Feedback -> Reservation -> Donor.

```
SELECT d.name, d.donor_type, d.area
FROM Donor d
WHERE d.donor_id IN (
    SELECT r.donor_id FROM Reservation r WHERE r.reservation_id IN (
        SELECT f.reservation_id FROM Feedback f WHERE f.rating = 5
    )
)
ORDER BY d.donor_id;
```

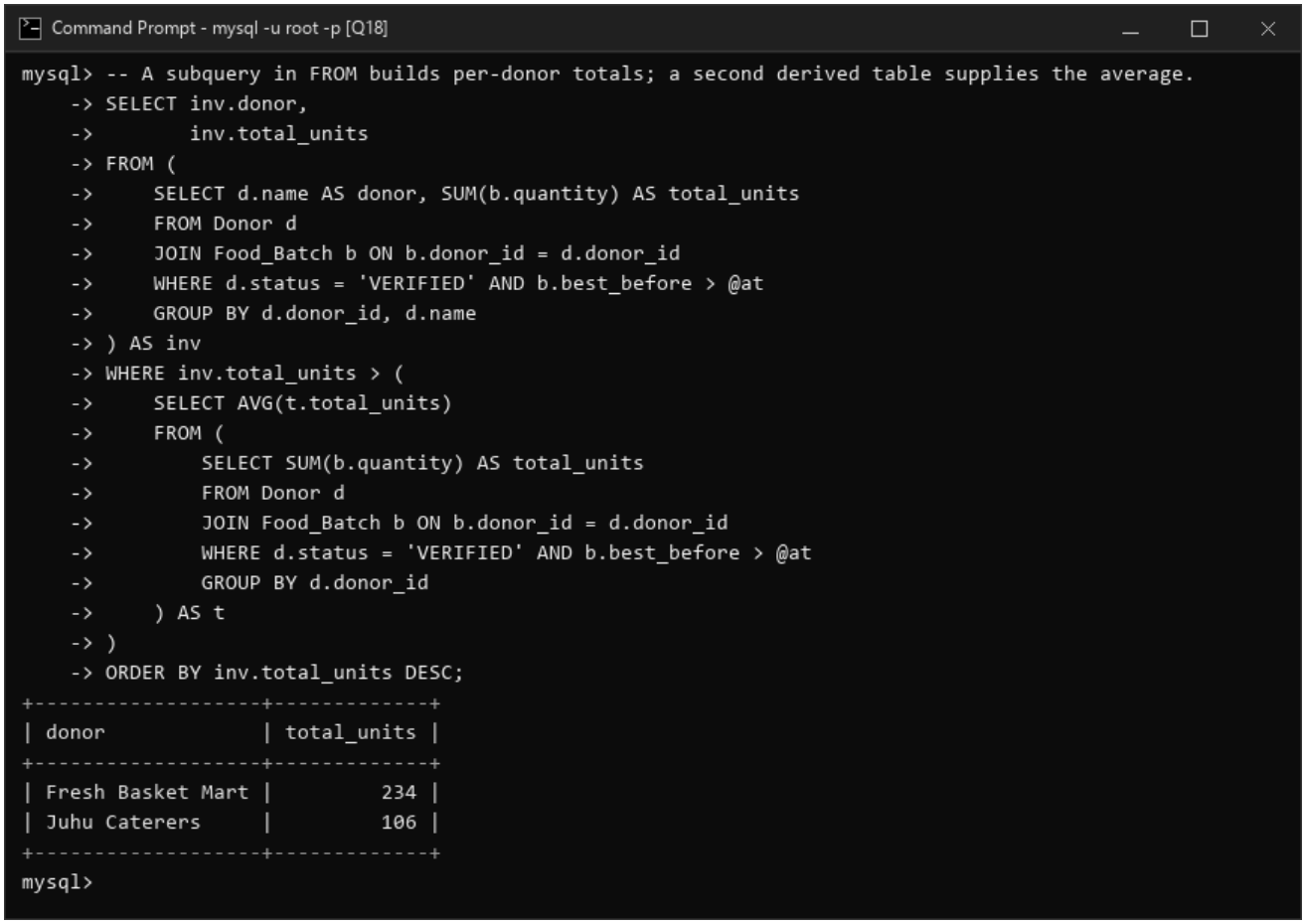


MySQL Output: Q17 execution result

Q18: Above-Average Stock Donors (Derived Table)

Objective: Identify donors holding unexpired stock quantities exceeding the overall donor average. **Explanation:** Subquery in FROM aggregates stock per donor; a second derived subquery in WHERE calculates the city-wide donor mean.

```
SELECT inv.donor, inv.total_units
FROM (
    SELECT d.name AS donor, SUM(b.quantity) AS total_units
    FROM Donor d JOIN Food_Batch b ON b.donor_id = d.donor_id
    WHERE d.status = 'VERIFIED' AND b.best_before > @at
    GROUP BY d.donor_id, d.name
) AS inv
WHERE inv.total_units > (
    SELECT AVG(t.total_units) FROM (
        SELECT SUM(b.quantity) AS total_units
        FROM Donor d JOIN Food_Batch b ON b.donor_id = d.donor_id
        WHERE d.status = 'VERIFIED' AND b.best_before > @at
        GROUP BY d.donor_id
    ) AS t
)
ORDER BY inv.total_units DESC;
```

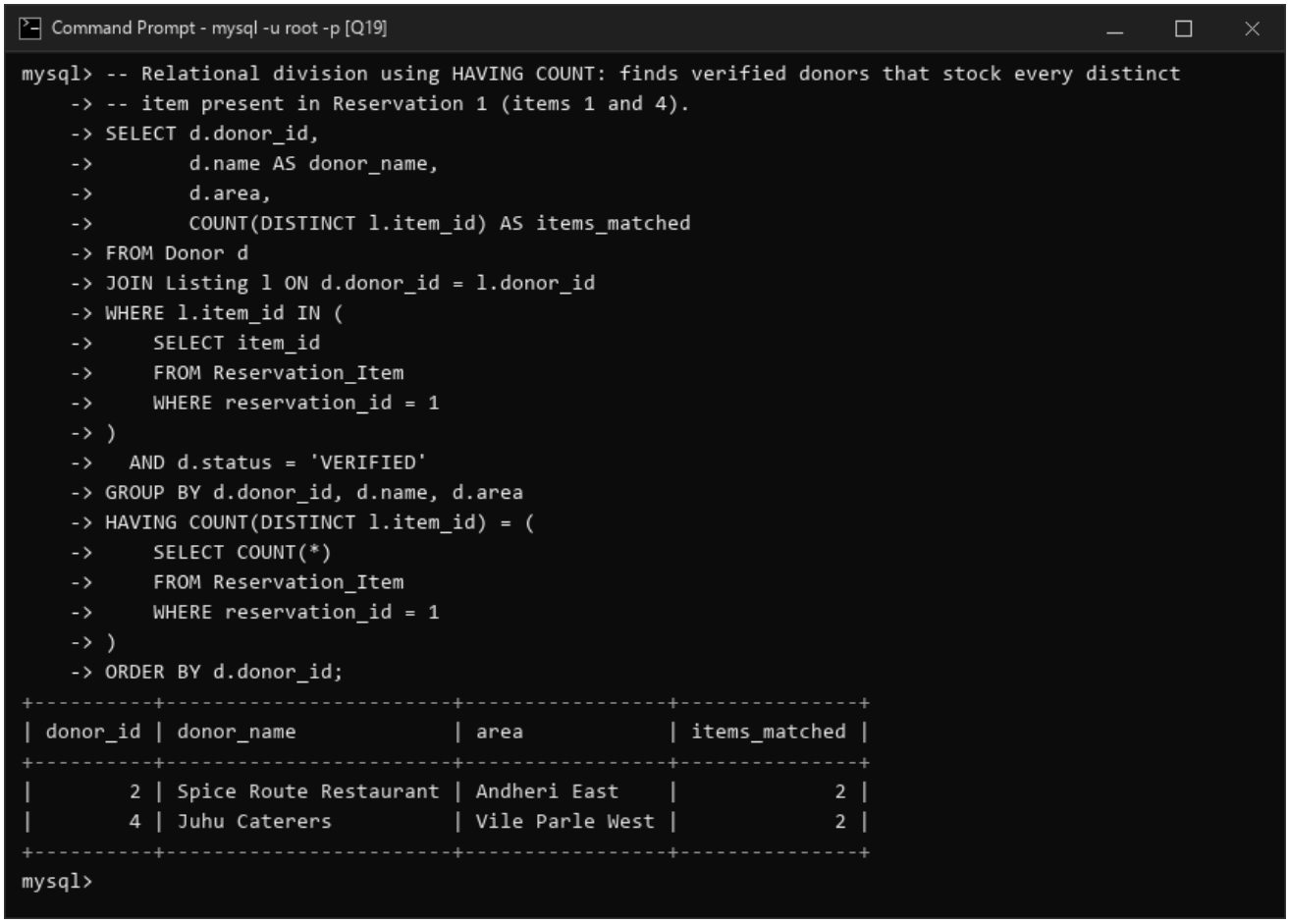


MySQL Output: Q18 execution result

Q19: Bulk Order Refill (Relational Division via HAVING COUNT)

Objective: Find which verified donors can supply ALL food items requested in bulk Reservation #1 (items 1 and 4). **Explanation:** Implements relational division using GROUP BY and HAVING COUNT to find donors whose count of matched distinct items equals the total item count in Reservation #1.

```
SELECT d.donor_id,
       d.name AS donor_name,
       d.area,
       COUNT(DISTINCT l.item_id) AS items_matched
FROM Donor d
JOIN Listing l ON d.donor_id = l.donor_id
WHERE l.item_id IN (
    SELECT item_id
    FROM Reservation_Item
    WHERE reservation_id = 1
)
  AND d.status = 'VERIFIED'
GROUP BY d.donor_id, d.name, d.area
HAVING COUNT(DISTINCT l.item_id) = (
    SELECT COUNT(*)
    FROM Reservation_Item
    WHERE reservation_id = 1
)
ORDER BY d.donor_id;
```

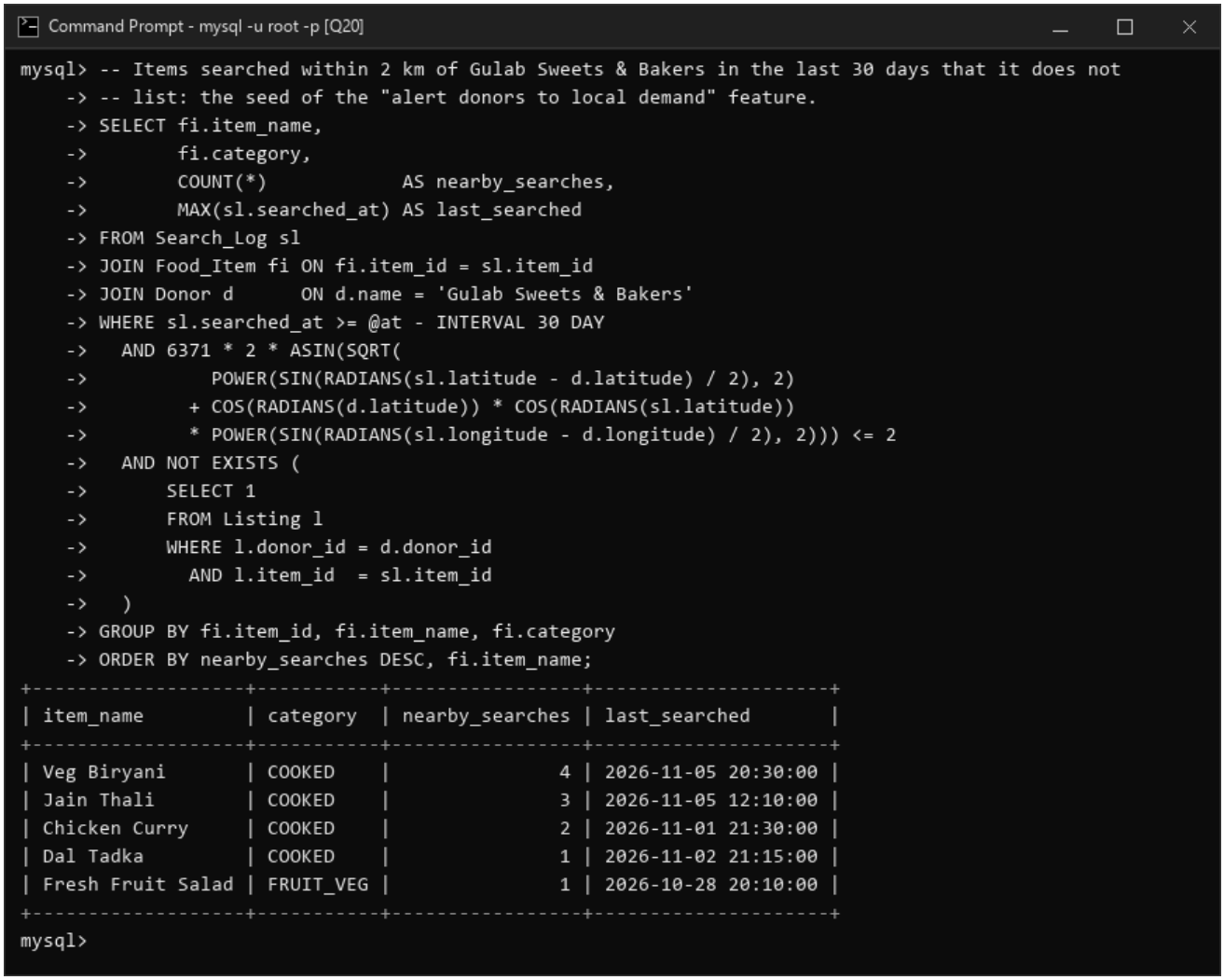


MySQL Output: Q19 execution result

Q20: Neighborhood Demand Intelligence (Correlated Geo-Subquery)

Objective: Identify items searched within 2 km of 'Gulab Sweets & Bakers' that the bakery does NOT currently stock. **Explanation:** Combines geospatial Haversine proximity on Search_Log with a NOT EXISTS anti-join on Listing to surface latent commercial demand.

```
SELECT fi.item_name, fi.category, COUNT(*) AS nearby_searches, MAX(sl.searched_at) AS last_searched
FROM Search_Log sl
JOIN Food_Item fi ON fi.item_id = sl.item_id
JOIN Donor d ON d.name = 'Gulab Sweets & Bakers'
WHERE sl.searched_at >= @at - INTERVAL 30 DAY
  AND 6371 * 2 * ASIN(SQRT(
         POWER(SIN(RADIANS(sl.latitude - d.latitude) / 2), 2)
       + COS(RADIANS(d.latitude)) * COS(RADIANS(sl.latitude))
       * POWER(SIN(RADIANS(sl.longitude - d.longitude) / 2), 2))) <= 2
  AND NOT EXISTS (
      SELECT 1 FROM Listing l WHERE l.donor_id = d.donor_id AND l.item_id = sl.item_id
  )
GROUP BY fi.item_id, fi.item_name, fi.category
ORDER BY nearby_searches DESC, fi.item_name;
```



MySQL Output: Q20 execution result